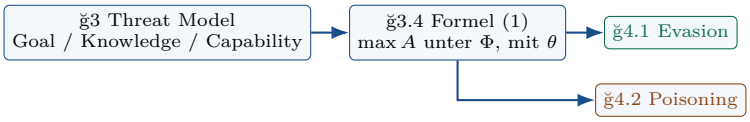
AML Cheat Sheet — Wild Patterns

Definitionen nach Paper-Sektionen & Verbindungen

Biggio & Roli (2018). Nur Paper-Inhalt. Fette Begriffe = zentrale Definitionen.

Die Verbindung in einem Satz

ğ3 definiert den Angreifer (Goal · Knowledge · Capability). Formel (1) verbindet die drei Achsen. ğ4 füllt (1) als **Evasion** (Test) oder **Poisoning** (Training) aus — jeweils generic/specific.



1 §3 Know Your Adversary — das Gerüst

Jedes Angriffsszenario wird durch **drei Dinge** festgelegt. Die Angriffe in ğ4 sind nur Ausformulierungen davon.

1.1 3.1 Attacker's Goal — Was will er?

Security Violation	*Integrity:* Detection umgehen, Normalbetrieb bleibt. *Availability:* System für Nutzer stören / DoS. *Privacy:* Infos über System/Nutzer/Daten (Reverse Engineering).
Attack Specificity	*targeted:* bestimmte Samples/Nutzer. *indiscriminate:* beliebige/alle Samples.
Error Specificity	*specific:* feste falsche Zielklasse. *generic:* irgendeine falsche Klasse reicht.

Verbindung: Goal sagt nur das Ziel — noch nicht *wie* oder *mit welchem Wissen*.

1.2 3.2 Attacker's Knowledge — Was weiß er?

Wissen: $\theta = (D, \mathcal{X}, f, w)$ (Trainingsdaten, Features, Algorithmus, Parameter).

Perfect-Knowledge (PK) White-Box	$\theta_{\text{PK}} = (D, \mathcal{X}, f, w)$ — alles bekannt; Worst-Case-Evaluation.
Limited-Knowledge (LK) Gray-Box	Teilwissen. Typisch: $\mathcal{X}$ und Art von f bekannt, nicht D, nicht w.
LK-SD	Surrogate-Daten: $\theta = (\hat{D}, \mathcal{X}, f, \hat{w})$.
LK-SL	Surrogate-Learner: $\theta = (\hat{D}, \mathcal{X}, \hat{f}, \hat{w})$.
Zero-Knowledge (ZK) Black-Box	$\theta_{\text{ZK}} = (\hat{D}, \hat{\mathcal{X}}, \hat{f}, \hat{w})$; praktisch Queries (Labels/Scores). Paper: Feature-Wissen kann partiell sein, aber nie völlig null (sonst keine sinnvollen Transformationen). Bilder: Features = Pixel.

Verbindung: Knowledge ändert nicht Evasion↔Poisoning, sondern *gegen welches Modell* optimiert wird (f oder Surrogat $\hat{f}$).

1.3 3.3 Attacker's Capability — Was darf er ändern?

Attack Influence	*exploratory* = nur Testdaten ⇒ **Evasion**. *causative* = Training manipulieren ⇒ **Poisoning**.
Data Manipulation Constraints	Angriffsdaten D_c nur in $\Phi(D_c)$ änderbar (z. B. Malware bleibt funktional; Distanzbounds; Features nur erhöhen).

Verbindung: Hier entsteht die Hauptgabel Evasion vs. Poisoning. Φ begrenzt jede Optimierung.

1.4 3.4 Attack Strategy — die gemeinsame Formel

$$D_c^\star \in \arg \max_{D_c' \in \Phi(D_c)} A(D_c', \theta) \tag{1}$$

Symbol	Bedeutung	kommt aus
A	Angriffserfolg	Goal
θ	Wissen / (Surrogat-)Modell	Knowledge
Φ	erlaubte Änderungen	Capability / Constraints

Verbindung: Goal + Knowledge + Capability stecken in (1). ğ4 füllt A und Φ konkret aus.

1.5 3.5 Security Evaluation Curves

Kein neuer Angriffstyp: Performance bei steigender **Attack Strength** (größeres Φ: mehr Perturbation / mehr Poison-Punkte).

1.6 3.6 Summary / Fig. 4 — Namen zuordnen

Kreuze Capability (Zeile) × Goal (Spalte):

	Integrity	Availability	Privacy
Test data	Evasion (adversarial examples)	—	Model extraction/stealing; model inversion
Training data	Poisoning für Intrusions		
(Backdoors / Trojans)	Poisoning (Fehler max.)	—	

Backdoor = **Poisoning + Integrity**, kein dritter Grundtyp neben Evasion/Poisoning.

2 §4 Simulating Attacks — Formel (1) ausfüllen

2.1 4.1 Evasion attacks (Test-Zeit)

Definition: Inputs manipulieren, um einen *bereits trainierten* Klassifikator *zur Test-Zeit* zu täuschen.
Connection zu ğ3: exploratory Capability; A oft auf einem x'; wahre Klasse k nur als Referenz in A (kein mitgeschicktes Trainingslabel).
Notation: $f_i(x)$ = Confidence Klasse i.

Error-generic Evasion	$\max_{x'} \Omega(x') = \max_{l \neq k} f_l(x') - f_k(x')$ s.t. $d(x, x') \leq d_{\max}$, $x_{\text{lb}} \preceq x' \preceq x_{\text{ub}}$ (2)–(3) Idee: nächste falsche Klasse hoch, richtige runter. Constraints = Φ.
Error-specific Evasion	wie (2)–(3), aber $A = -\Omega$ und f_k = Confidence der **Zielklasse**.

2.2 4.2 Poisoning Attacks (Training)

Definition: Giftpunkte in *Trainingsdaten* ⇒ manipuliertes $w^\star$; mehr Fehler zur *späteren* Test-Zeit.
Connection zu ğ3: causative Capability; Bilevel: innen Lernen, außen A.

Error-generic Poisoning	$\max_{D'_c \in \Phi} A = L(D_{\text{val}}, w^\star)$ s.t. $w^\star \in \arg\min_{w'} L(D_{\text{tr}} \cup D'_c, w')$ (4)–(5) Viele Fehler, Klasse egal (DoS/Availability).
Error-specific Poisoning	weiter (4)–(5), aber $A = -L(D'_{\text{val}}, w^\star)$ mit Angreifer-Labels.
Backdoor / Trojan	Poisoning Integrity: gezielte spätere Fehlklassifikation (Trigger/Region).

3 Gegenüberstellung & Entscheidungsbaum

	Evasion	Poisoning
Capability	exploratory / Test	causative / Training
Variable	meist x'	Menge D'_c
A **generic**	$\max_{l \neq k} f_l - f_k$	$L(D_{\text{val}}, w^\star)$
A **specific**	$-\Omega$ / Zielklasse	$-L(D'_{\text{val}}, w^\star)$
Knowledge	PK/LK/ZK (f oder $\hat{f}$)	ebenso

4 Fragen ⇒ Angriff einordnen

1. Training? Ja → Poisoning. Nein → Evasion.
2. Beliebige Fehler oder feste Klasse? → error-generic / specific.
3. Wissen über $D, \mathcal{X}, f, w$? → PK / LK / ZK.
4. Erlaubte Änderungen? → $\Phi(D_c)$.

Begriffspaare — nicht vermischen

Evasion ↔ Poisoning = *wann.* generic ↔ specific = *welche Fehlklassifikation.* targeted ↔ indiscriminate = *welche Samples.* Integrity ↔ Availability ↔ Privacy = *Security-Ziel.* PK/LK/ZK = *Wissen* (orthogonal).

Quelle: B. Biggio, F. Roli. *Wild Patterns.* Pattern Recognition 84:317–331, 2018 (Abschnitte 3–4; Formeln (1)–(5); Fig. 1, 4, 5).